

# Problem Set 5 Lab Report: Investing for Retirement

## Overview

Problem Set 5 focused on retirement investment planning using successive approximation. The assignment required four Python programs that calculated retirement account growth, variable investment returns, retirement withdrawals, and the maximum annual retirement expense that could reduce the account balance close to zero. The assignment demonstrated program efficiency, iteration, list processing, function reuse, and binary search.

The four programs were designed to build on one another. The first program calculated account growth using a fixed annual return rate. The second program expanded the model by allowing annual return rates to change over time. The third program modeled retirement withdrawals after the account had already accumulated savings. The fourth program used binary search to estimate the maximum annual withdrawal amount.

## Problem One: Fixed Investment Growth

The first program created a function named `fixedInvestor()`. This function calculated the value of a retirement account at the end of each year using salary, employee savings rate, fixed growth rate, and number of years. The model assumed that the employer contributes 5% of salary and matches the employee contribution up to an additional 5%.

The function used annual compounding. For the first year, the retirement account value was based only on the annual contribution because no prior balance existed to earn interest. In later years, the previous account balance earned interest, and the new annual contribution was added.

This program demonstrated successive approximation because the account value was updated year by year. Each year's value depended on the prior year's ending balance.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "/TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberlands/MSDS-532/assignment_5_1.py"
[7500.000000000001, 15375.000000000002, 23643.750000000004]
```

## Problem Two: Variable Investment Growth

The second program created a function named `variableInvestor()`. This function expanded the first problem by replacing the fixed annual growth rate with a list of yearly growth rates. The length of the list determined how many years were evaluated.

This version better represented real investment behavior because returns are rarely identical every year. Each year's account value was calculated using the growth rate for that year. The function returned a list of yearly ending balances.

This problem demonstrated list iteration and flexible modeling. Instead of assuming a single constant rate, the function used a sequence of rates to approximate changing investment performance over time.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "/:/TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_5_2.py"
[7500.000000000001, 15375.000000000002, 23490.000000000004]
```

## Problem Three: Retirement Withdrawals

The third program created a function named `finallyRetired()`. This function modeled what happens after retirement when the account already has a starting balance. Unlike the earlier investment stage, interest was earned every year because the account began with established principal.

Each year, the account earned interest based on the annual rate of return, and then a fixed amount was withdrawn. The function returned a list showing the remaining account value after each year.

This problem demonstrated how financial modeling changes after retirement. During the working years, the account grows through contributions and returns. During retirement, the account may still earn returns, but withdrawals reduce the balance.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "/:/TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_5_3.py"
[95000.0, 88800.0, 81464.0]
```

## Problem Four: Maximum Annual Expense

The fourth program created a function named `maximumExpensed()`. This function used binary search to estimate the maximum annual retirement withdrawal that would leave the account balance close to zero after the retirement period.

The function first used `variableInvestor()` to calculate the amount saved by retirement. It then used `finallyRetired()` to test different annual withdrawal values. Binary search was used because it is more efficient than testing every possible withdrawal amount one by one. If the remaining balance was too high, the withdrawal estimate increased. If the remaining balance was negative, the estimate decreased.

The search continued until the remaining balance was within the epsilon value provided by the user. Each time the withdrawal estimate changed, the program printed the annual expense amount and remaining account value.

This problem demonstrated efficiency and order-of-growth concepts. Binary search reduces the number of guesses by repeatedly cutting the search range in half, making it much more efficient than a linear search.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe -u -m TRUENAS/ShadowRealm/Alan's Prometheus B
ackup/Education/US College/University of the Cumberland/MSDS-532/assignment5_4.py"
Trying an annual expense of $20435.69, the remaining value is $-60079.93.
Trying an annual expense of $10217.85, the remaining value is $-6581.61.
Trying an annual expense of $5108.92, the remaining value is $20167.54.
Trying an annual expense of $7663.38, the remaining value is $6792.96.
Trying an annual expense of $8940.61, the remaining value is $105.68.
Trying an annual expense of $9579.23, the remaining value is $-3237.97.
Trying an annual expense of $9259.92, the remaining value is $-1566.15.
Trying an annual expense of $9100.27, the remaining value is $-730.24.
Trying an annual expense of $9020.44, the remaining value is $-312.28.
Trying an annual expense of $8980.53, the remaining value is $-103.30.
Trying an annual expense of $8960.57, the remaining value is $1.19.
Trying an annual expense of $8970.55, the remaining value is $-51.06.
Trying an annual expense of $8965.56, the remaining value is $-24.94.
Trying an annual expense of $8963.07, the remaining value is $-11.87.
Trying an annual expense of $8961.82, the remaining value is $-5.34.
Trying an annual expense of $8961.20, the remaining value is $-2.08.
Trying an annual expense of $8960.88, the remaining value is $-0.45.
Trying an annual expense of $8960.73, the remaining value is $0.37.
Trying an annual expense of $8960.81, the remaining value is $-0.04.
Trying an annual expense of $8960.77, the remaining value is $0.17.
Trying an annual expense of $8960.79, the remaining value is $0.06.
Trying an annual expense of $8960.80, the remaining value is $0.01.
Trying an annual expense of $8960.80, the remaining value is $-0.01.
Final annual expense estimate is $8960.80, leaving $0.00.
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code>
```

## Conclusion

Problem Set 5 demonstrated how Python can be used to model retirement planning scenarios through successive approximation. The assignment applied functions, loops, lists, compounding calculations, and binary search. Each program built on the previous one, showing how modular programming allows earlier solutions to be reused in more complex problems.

The first two programs modeled retirement savings during working years. The third program modeled withdrawals during retirement. The fourth program combined the earlier functions and used binary search to estimate the maximum sustainable annual retirement expense. Overall, the assignment reinforced the importance of efficiency, decomposition, and reusable function design in programming.